

Tarea 2: Aprendizaje Distribuido para Computer Vision

Clasificación de Maravillas del Mundo con JAX y Flax NNX

Curso: IIC2026

Profesor: Johansell Villalobos Cubillo

Estudiante: Maria Fernanda Moroney Sole

Fecha: Junio 2026

1. Entorno de ejecución

Todos los experimentos se ejecutaron en Google Colab utilizando una GPU NVIDIA T4 como acelerador, con JAX sobre el backend CUDA. Las dependencias se instalaron con `pip install -U`

`"jax[cuda12]" flax optax kagglehub`, lo que garantiza versiones compatibles entre JAX y el plugin de GPU. La verificación de dispositivos produjo la siguiente salida:

```
Dispositivos: [CudaDevice(id=0)]
Cantidad de dispositivos: 1
Backend: gpu
```

Acelerador documentado para todos los experimentos: **1 × GPU NVIDIA T4 (CUDA)**. Se adjunta además la captura de pantalla del entorno de Colab mostrando el acelerador detectado.

```
import jax
import jax.numpy as jnp
from flax import nnx
import optax
import numpy as np
import time
import matplotlib.pyplot as plt

print("Dispositivos:", jax.devices())
print("Cantidad de dispositivos:", jax.device_count())
print("Backend:", jax.default_backend())

Dispositivos: [CudaDevice(id=0)]
Cantidad de dispositivos: 1
Backend: gpu
```

2. Dataset y preprocesamiento

Se utilizó el conjunto *Wonders of the World Image Classification* (Kaggle), descargado automáticamente mediante `kagglehub`. Contiene **3.846 imágenes** distribuidas en **12 clases**: `burj_khalifa` (390), `chichen_itza` (340), `christ_the_reedemer` (323), `eiffel_tower` (391), `great_wall_of_china` (392), `machu_pichu` (393), `pyramids_of_giza` (372), `roman_colosseum` (394), `statue_of_liberty` (238), `stonehenge` (204), `taj_mahal` (158) y `venezuela_angel_falls` (251). El conjunto está moderadamente desbalanceado (158–394 imágenes por clase).

Decisiones de preprocesamiento adoptadas:

Las imágenes se redimensionaron a **64×64 píxeles RGB**, un balance entre el detalle necesario para distinguir monumentos y el costo computacional en una sola GPU. Los valores de píxel se normalizaron al rango [0,1] dividiendo entre 255, lo que estabiliza el entrenamiento. Se construyó un **split estratificado por clase 70/15/15** con semilla fija (42) para reproducibilidad, resultando en **2.686 imágenes de entrenamiento, 570 de validación y 590 de prueba**. Las imágenes corruptas se descartan durante la carga. Todo el conjunto se mantiene en memoria como arrays de NumPy y se alimenta al acelerador mediante un generador de lotes con barajado por época, un pipeline simple y eficiente para un dataset de este tamaño.

3. Arquitectura del modelo

El modelo se implementó exclusivamente con **Flax NNX**, derivando de `nnx.Module` y siguiendo la estructura mínima exigida, ampliada con normalización por lotes:

```
Conv(3→f1, 3x3) → BatchNorm → ReLU → MaxPool(2x2)
Conv(f1→f2, 3x3) → BatchNorm → ReLU → MaxPool(2x2)
Flatten → Linear(dense) → ReLU → Linear(12) [logits]
```

Los hiperparámetros estructurales (f1, f2, dense) son configurables para los experimentos de tamaño de red, y el parámetro `dtype` controla la precisión numérica de cómputo (los logits finales se proyectan a float32 por estabilidad del softmax). La configuración base (f1=32, f2=64, dense=256) tiene **4.217.420 parámetros (16,9 MB)**. El entrenamiento usa diferenciación automática (`nnx.value_and_grad`), pérdida cross-entropy con etiquetas enteras, optimizador **Adam** (Optax) y compilación JIT mediante `nnx.jit`.

4. Entrenamiento base

Configuración: 15 épocas, batch=64, lr=10⁻³, float32. Resultados:

Métrica	Valor
Tiempo total de entrenamiento	11,9 s
Tiempo promedio por época	0,67 s
Throughput	3.921 img/s
Exactitud de entrenamiento	0,999
Exactitud de validación	0,766
Exactitud de prueba	0,799

La primera época tardó 3,64 s frente a ~0,46 s las restantes: esa diferencia corresponde al costo de compilación JIT, que solo se paga una vez. Las curvas muestran un claro **sobreajuste**: el modelo alcanza 99,9% en entrenamiento pero ~77% en validación, lo esperable para un dataset pequeño sin aumento de datos.

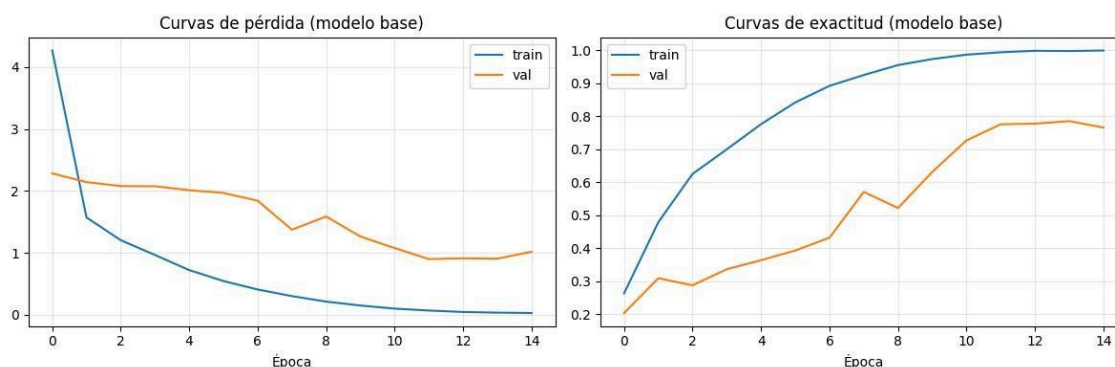


Figura 1. Curvas de pérdida y exactitud del modelo base (15 épocas).

5. Experimentos

5.1 Impacto del tamaño de lote (10 épocas)

Batch	Tiempo total (s)	Tiempo/época (s)	Throughput (img/s)	Exactitud test
32	9,6	0,93	2.863	0,811
64	7,2	0,69	3.821	0,670
128	7,3	0,71	3.625	0,393

El lote de 32 obtuvo la mejor exactitud (0,811) a costa del menor throughput; el lote de 128 fue el peor en exactitud (0,393). Con el número de épocas fijo, un lote mayor implica muchas menos actualizaciones de gradiente por época (21 con bs=128 frente a 83 con bs=32), por lo que el modelo simplemente entrena menos. En throughput, bs=64 aprovechó mejor la GPU que bs=32, confirmando que lotes mayores aumentan el paralelismo del acelerador.

5.2 Impacto de la tasa de aprendizaje (10 épocas)

Learning rate	Pérdida final (val)	Acc. val	Acc. test	Tiempo total (s)
10^{-2}	2,2817	0,197	0,180	6,8
10^{-3}	1,2552	0,656	0,666	6,7
10^{-4}	1,0504	0,715	0,709	7,3

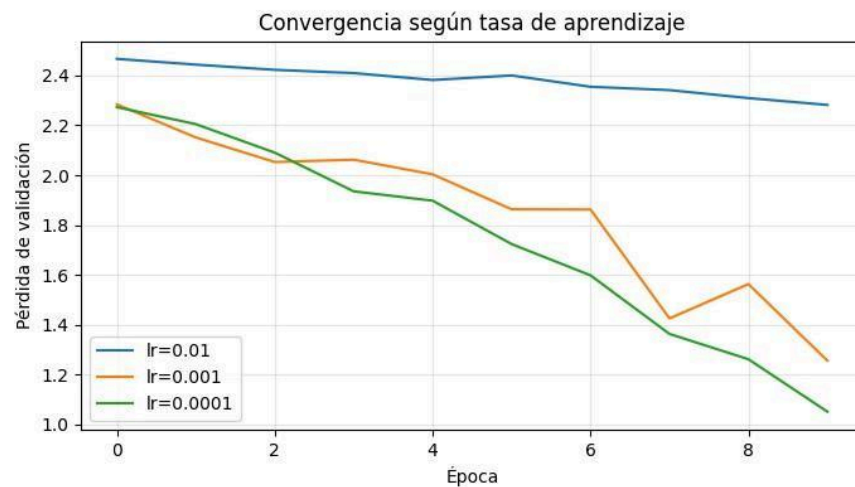


Figura 2. Convergencia de la pérdida de validación según la tasa de aprendizaje.

Con $lr=10^{-2}$ el modelo prácticamente no converge (la pérdida se estanca cerca del valor inicial): el paso es demasiado grande y el optimizador oscila. $lr=10^{-3}$ converge rápido pero con cierta inestabilidad, y $lr=10^{-4}$ converge de forma más suave y alcanzó la mejor exactitud en 10 épocas, aunque con más épocas $lr=10^{-3}$ tendería a alcanzarla antes. La tasa de aprendizaje resultó el hiperparámetro más sensible para la exactitud final.

5.3 Impacto del tamaño de la red (8 épocas)

Filtros (f1,f2)	Dens e	Tiempo (s)	Throughput (img/s)	Acc. test
(32, 64)	128	6,1	3.856	0,498
(32, 64)	256	6,1	3.594	0,598
(32, 64)	512	6,8	3.478	0,562
(64, 128)	128	10,7	2.248	0,434
(64, 128)	256	9,4	2.431	0,557
(64, 128)	512	9,9	2.307	0,512
(128, 256)	128	19,0	1.260	0,504
(128, 256)	256	17,0	1.316	0,400
(128, 256)	512	17,7	1.265	0,531

Aumentar la cantidad de filtros multiplicó el tiempo de entrenamiento (de ~6 s a ~18 s, con el throughput cayendo de ~3.800 a ~1.300 img/s) **sin mejorar la exactitud**: con solo 8 épocas y un dataset pequeño, los modelos grandes no alcanzan a explotar su capacidad y tienden a sobreajustar. La mejor combinación fue la más ligera, (32,64) con 256 neuronas (0,598).

6. Optimización de hiperparámetros (grid search)

Se realizó una búsqueda en grilla de 16 configuraciones ($lr \in \{10^{-3}, 3 \cdot 10^{-4}\} \times batch \in \{64, 128\} \times filtros \in \{(32,64), (64,128)\} \times dense \in \{256, 512\}$), con 8 épocas por configuración. Las 5 mejores por exactitud de prueba:

Configuración	Acc. test	Pérdida final	Tiempo (s)	Throughput (img/s)
bs=64, lr=1e-3, f=(32,64), d=512	0,580	1,3437	6,0	3.662
bs=64, lr=1e-3, f=(64,128), d=256	0,572	1,3788	9,4	2.310
bs=64, lr=3e-4, f=(32,64), d=512	0,572	1,5152	6,0	3.656
bs=64, lr=1e-3, f=(32,64), d=256	0,564	1,4419	6,5	3.362
bs=64, lr=3e-4, f=(64,128), d=256	0,553	1,4603	10,1	2.152

Patrones claros: **todas** las configuraciones con batch=64 superaron a las de batch=128, y las redes ligeras (32,64) igualaron o superaron a las grandes con la mitad o un tercio del tiempo. Combinando exactitud, tiempo y consumo de recursos, la configuración con mejor balance fue **batch=64, lr=10⁻³, filtros=(32,64), dense=512, float32** (0,580 de exactitud en solo 6,0 s). Cabe señalar que estas exactitudes corresponden a 8 épocas; con el presupuesto del modelo base (15 épocas) esta misma configuración alcanza ~0,80 de exactitud de prueba.

7. Análisis de precisión numérica

Sobre la configuración base se varió únicamente el tipo de dato de cómputo (10 épocas):

dtype	Acc. test	Pérdida final	Tiempo (s)	Throughput (img/s)	Memoria pico (MB)
float32	0,641	1,3182	7,9	3.424	2.902
float16	0,703	1,1405	10,0	3.332	2.902
bfloat16	0,637	1,2678	8,8	3.503	2.902

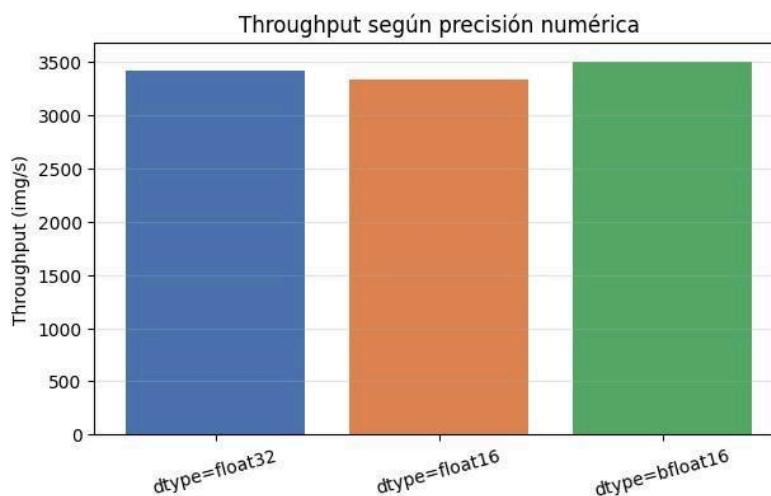


Figura 3. Throughput según precisión numérica.

Los tres formatos completaron el entrenamiento de forma **numéricamente estable** (sin NaN ni divergencias), en parte porque los logits y la pérdida se mantienen en float32. Las diferencias de exactitud entre formatos (0,637–0,703) están dentro de la variabilidad normal entre corridas para un modelo de este tamaño. Contrario a lo esperado, la precisión reducida **no aceleró** el entrenamiento: el modelo es tan pequeño que el cuello de botella es el lanzamiento de kernels y la transferencia de datos, no el cómputo, y además los Tensor Cores de la T4 se aprovechan más con modelos grandes. La memoria pico reportada fue idéntica (2.902 MB) porque JAX **prealoca** un porcentaje fijo de la VRAM, lo que constituye una limitación de medición de `memory_stats()` más que una ausencia real de ahorro. En modelos grandes, float16/bfloat16 sí reducen memoria y aumentan rendimiento; bfloat16 es preferible a float16 por conservar el rango de exponente de float32 (menor riesgo de overflow/underflow).

8. Análisis y discusión

1. ¿Qué ventajas ofrece Flax NNX respecto a implementar modelos directamente en JAX?

NNX administra automáticamente el estado del modelo (parámetros, estadísticas de BatchNorm, generadores aleatorios) con una sintaxis orientada a objetos. En JAX puro habría que crear, pasar y actualizar manualmente estructuras de parámetros en cada función pura, lo cual es verboso y propenso a errores. NNX además provee capas listas (Conv, Linear, BatchNorm), modos train/eval y transformaciones conscientes del estado como `nnx.jit` y `nnx.value_and_grad`.

2. ¿Qué beneficios aporta la compilación mediante `jax.jit`?

Compila el grafo de operaciones con XLA: fusiona operaciones, optimiza memoria y elimina el overhead del intérprete de Python. Se observó directamente: la primera época tardó 3,64 s (compilación) y las siguientes ~0,46 s, una aceleración de ~8× en estado estacionario.

3. ¿Qué hiperparámetro tuvo mayor impacto sobre la exactitud final?

La tasa de aprendizaje: entre $lr=10^{-2}$ y $lr=10^{-4}$ la exactitud de prueba pasó de 0,180 a 0,709, la mayor brecha observada en todos los experimentos. Un lr inadecuado impide por completo la convergencia, mientras que los demás hiperparámetros solo modulan el resultado.

4. ¿Qué hiperparámetro tuvo mayor impacto sobre el tiempo de entrenamiento?

El tamaño de la red (número de filtros convolucionales): pasar de filtros (32,64) a (128,256) triplicó el tiempo total (de ~6 s a ~18 s) y redujo el throughput de ~3.800 a ~1.300 img/s.

5. ¿Cómo afectó el tamaño de lote al rendimiento observado?

Lotes mayores mejoraron el uso del acelerador (throughput de 2.863 img/s con $bs=32$ a 3.821 con $bs=64$), pero con épocas fijas redujeron la exactitud (0,811 $\rightarrow$ 0,393 entre $bs=32$ y $bs=128$) por realizar menos actualizaciones de gradiente. Existe un compromiso entre eficiencia de hardware y calidad de optimización; $bs=64$ resultó el mejor punto medio.

6. ¿Qué diferencias encontró entre float32, float16 y bfloat16?

Los tres fueron estables y con exactitud comparable (0,637–0,703). No hubo ganancia de velocidad ni de memoria medible porque el modelo es demasiado pequeño para que el cómputo sea el cuello de botella y porque la preasignación de memoria de JAX enmascara el ahorro. Conceptualmente, float16 ofrece más mantisa pero poco rango (riesgo de overflow); bfloat16 conserva el rango de float32 con menos precisión, por lo que es la opción de 16 bits más segura para entrenar.

7. ¿Cuál configuración produjo el mejor balance entre rendimiento y exactitud?

$batch=64$, $lr=10^{-3}$, $filtros=(32,64)$, $dense=512$, float32: 0,580 de exactitud (8 épocas) en solo 6,0 s y 3.662 img/s. Entrenada con 15 épocas como el modelo base, esta familia de configuraciones alcanza ~0,80 de exactitud de prueba manteniendo el menor costo computacional de la grilla.

8. ¿Qué limitaciones encontró al utilizar Google Colab?

Incompatibilidad inicial entre la versión de JAX y el plugin CUDA preinstalado (error PJRT), que obligó a reinstalar `jax[cuda12]` y reiniciar la sesión; tiempo de sesión limitado y riesgo de desconexión durante experimentos largos; GPU compartida con rendimiento variable; RAM y VRAM limitadas; y la preasignación de memoria de JAX que impide medir el consumo real por configuración.

9. ¿Qué mejoras futuras propondría para aumentar el desempeño del modelo?

Aumento de datos (rotaciones, recortes, volteos) para combatir el sobreajuste observado (train 0,999 vs. val 0,766); regularización adicional (dropout, weight decay); arquitecturas más profundas con conexiones residuales (ResNet); learning rate schedules (cosine decay, warmup); early stopping sobre la validación; transfer learning desde modelos preentrenados; mayor resolución de entrada; y, con múltiples dispositivos, paralelismo de datos con sharding de JAX.

9. Conclusiones

Se implementó y entrenó exitosamente un clasificador convolucional de 12 maravillas del mundo con JAX y Flax NNX sobre una GPU T4, alcanzando **79,9% de exactitud de prueba** con el modelo base de 4,2 M de parámetros en menos de 12 segundos de entrenamiento. La compilación JIT demostró un impacto decisivo (~8× de aceleración tras la primera época) y el ecosistema NNX simplificó notablemente la gestión del estado del modelo. Los experimentos mostraron que, en este régimen de pocos datos y pocas épocas, la tasa de aprendizaje domina la exactitud, el tamaño de la red domina el costo, y los lotes moderados (64) equilibran throughput y calidad de optimización. La precisión reducida no aportó beneficios medibles en un modelo tan pequeño, evidenciando que sus ventajas dependen de que el cómputo sea el cuello de botella. El principal límite del modelo es el sobreajuste, abordable con aumento de datos y regularización como trabajo futuro.